

Proposed Dashboard — File Structure

Date: June 30, 2026

This document proposes a modular file structure for the Dashboard view, following the same organizational pattern established by the Inventory view. Includes all modals and their trigger logic.

1. Current State (Before Refactoring)

The entire Dashboard currently lives in a single 457-line file:

```
src/
  ■■■ components/
  ■   ■■■ Dashboard.tsx ← 457 lines, everything in one file
  ■   ■■■ Inventory.tsx
  ■■■ shared/
  ■■■ inventory/      ← already modularized
```

2. Proposed File Structure (After Refactoring)

A new src/shared/dashboard/ folder mirroring the Inventory pattern:

```
src/
  ■■■ components/
  ■   ■■■ Dashboard.tsx ← thin orchestrator
  ■   ■■■ Inventory.tsx
  ■■■ shared/
  ■■■ inventory/
  ■■■ dashboard/ ← NEW
  ■■■ components/
  ■   ■■■ MetricsCards.tsx
  ■   ■■■ AssetsTable.tsx
  ■   ■■■ DonutChart.tsx
  ■   ■■■ RecentActivity.tsx
  ■   ■■■ PeopleSidebar.tsx
  ■   ■■■ AddAssetModal.tsx
  ■   ■■■ EditAssetModal.tsx
  ■   ■■■ DeleteAssetModal.tsx
  ■   ■■■ ReassignAssetModal.tsx
  ■   ■■■ AssetDetailModal.tsx
  ■■■ hooks/
  ■   ■■■ useAssets.ts
  ■   ■■■ useDashboardApi.ts
  ■   ■■■ usePeople.ts
  ■■■ constants.ts
```

3. File-by-File Breakdown

Display Components

- **MetricsCards.tsx** — Top row of four KPI cards (Total Assets, Active, Maintenance Due, Retired). Accepts a metrics prop.
- **AssetsTable.tsx** — Search bar, category/status dropdowns, data table, pagination footer. Accepts assets[] and filter callbacks.
- **DonutChart.tsx** — Reusable conic-gradient donut. Accepts segments[] and title. Used twice: Category and Status charts.
- **RecentActivity.tsx** — Activity timeline feed. Accepts activities[] with type, description, actor, timestamp.
- **PeopleSidebar.tsx** — Right-hand panel listing team members with avatars. Accepts people[] and onViewAll.

4. Modal Components & Their Logic

Each modal follows the same pattern used in the Inventory view: an isOpen boolean prop, an onClose callback, and a backdrop overlay with blur(4px).

4.1 AddAssetModal.tsx

Trigger: User clicks the green "+ Add Asset" button in the Assets table header.

Purpose: Form dialog to register a new asset into the system.

Form Fields:

- Asset Tag (auto-generated, read-only, e.g. AST-1009)
- Asset Name (text input, required)
- Category (dropdown: Laptop, Monitor, Printer, Server, Phone, Projector, Accessory, Furniture)
- Location (text input, required)
- Assigned To (dropdown populated from the People list)

Logic Flow:

- 1. User fills in the form fields.
- 2. On submit, useAssets hook sends a POST request to the Django backend.
- 3. On success: modal closes, asset list refreshes, metrics cards update counts.
- 4. On error: inline validation message appears below the failed field.

4.2 EditAssetModal.tsx

Trigger: User clicks the ■ (three-dot menu) on an asset row → selects "Edit".

Purpose: Pre-filled form to modify an existing asset's details.

Form Fields: Same as AddAssetModal, but pre-populated with the selected asset's current values.

Logic Flow:

- 1. The selected asset object is passed as a prop: asset: Asset.
- 2. Form fields are initialized with the asset's current data.

- 3. On submit, useAssets hook sends a PATCH request to /api/assets/:id/.
- 4. On success: modal closes, the row updates in-place, metrics recalculate if category/status changed.

4.3 DeleteAssetModal.tsx

Trigger: User clicks the  menu on an asset row → selects **"Delete"**.

Purpose: Confirmation dialog to prevent accidental deletion.

UI: Warning icon, asset name displayed prominently, two buttons: **Cancel** (secondary) and **Delete** (destructive red).

Logic Flow:

- 1. Receives the asset object as a prop.
- 2. Displays: "Are you sure you want to delete **AST-1004** (Dell PowerEdge T150)?"
- 3. On confirm, useAssets hook sends a DELETE request to /api/assets/:id/.
- 4. On success: modal closes, asset removed from list, Total Assets count decrements.

4.4 ReassignAssetModal.tsx

Trigger: User clicks the  menu on an asset row → selects **"Reassign"**.

Purpose: Quick-action modal to transfer an asset to a different team member without editing all fields.

Form Fields:

- Current Owner (read-only display, e.g. "Emma Johnson")
- New Owner (searchable dropdown populated from People list)
- Reason (optional text area for audit trail)

Logic Flow:

- 1. Receives asset and people[] as props.
- 2. On submit, sends PATCH to update only assignedTo field.
- 3. On success: modal closes, table row updates, a new entry appears in Recent Activity.

4.5 AssetDetailModal.tsx

Trigger: User clicks directly on an **asset row** in the table (not the  menu).

Purpose: Read-only detail view showing the full asset profile, similar to how the Inventory view shows workstation diagnostics.

Displayed Sections:

- Asset header: tag, name, status badge, category icon
- Details grid: Location, Assigned To, Date Added, Last Updated
- Maintenance history timeline (if any)
- Action buttons at the bottom: "Edit", "Reassign", "Delete" (which open their respective modals)

Logic Flow:

- 1. Receives the full asset object as a prop.
- 2. Renders a read-only detail card; no form submission.
- 3. The footer action buttons set state to open EditAssetModal, ReassignAssetModal, or DeleteAssetModal.

5. Modal Trigger Map

Summary of every user interaction that opens a modal:

Modal	UI Trigger	HTTP Method	Props Required
AddAssetModal	" + Add Asset" button	POST	isOpen, onClose, onSubmit, people[]
EditAssetModal	■ menu → Edit	PATCH	isOpen, onClose, asset, onSave
DeleteAssetModal	■ menu → Delete	DELETE	isOpen, onClose, asset, onConfirm
ReassignAssetModal	■ menu → Reassign	PATCH	isOpen, onClose, asset, people[], onSave
AssetDetailModal	Click asset row	GET (read-only)	isOpen, onClose, asset

6. Modal State Management in useDashboardApi.ts

The master hook manages which modal is open and which asset is selected:

```
// Inside useDashboardApi.ts
const [activeModal, setActiveModal] = useState<string | null>(null);
const [selectedAsset, setSelectedAsset] = useState<Asset | null>(null);

const openModal = (modalName: string, asset?: Asset) => {
  setActiveModal(modalName);
  if (asset) setSelectedAsset(asset);
};

const closeModal = () => {
  setActiveModal(null);
  setSelectedAsset(null);
};

// Modal name constants
// 'add-asset' | 'edit-asset' | 'delete-asset' | 'reassign-asset' | 'asset-detail'
```

The orchestrator (Dashboard.tsx) then renders each modal conditionally:

```
<AddAssetModal isOpen={activeModal === 'add-asset'} onClose={closeModal} ... />
<EditAssetModal isOpen={activeModal === 'edit-asset'} asset={selectedAsset} onClose={closeModal} ... />
<DeleteAssetModal isOpen={activeModal === 'delete-asset'} asset={selectedAsset} onClose={closeModal} ... />
<ReassignAssetModal isOpen={activeModal === 'reassign-asset'} asset={selectedAsset} onClose={closeModal} ... />
<AssetDetailModal isOpen={activeModal === 'asset-detail'} asset={selectedAsset} onClose={closeModal} ... />
```

7. Inventory vs. Dashboard — Modal Comparison

Purpose	Inventory Modal	Dashboard Modal
Add new item	AddDepartmentModal AddMemberModal	AddAssetModal
Edit existing item	EditSpecsModal	EditAssetModal
Delete item	(inside EditSpecsModal)	DeleteAssetModal
Reassign / transfer	—	ReassignAssetModal
View full detail	DiagnosticsGrid (inline)	AssetDetailModal

App.tsx remains completely untouched — it still imports <Dashboard /> from ./components/Dashboard. All modal logic is internal to the dashboard module.